

Pedix

Agentic Retrieval-Augmented Generation for
Pediatric Symptom-to-Care Pathway Navigation

Type: Cloud-Deployed AI Web Application

Domain: Pediatric Healthcare (Under-5 Triage)

Architecture: High Level Agentic RAG with Custom Reasoning Loop

Cloud Platform: Amazon Web Services (AWS)

Team Size: 2 Members

Github: github.com/Robertise/Pedix

Demo: [Pedix](#)

Member	Primary Contribution	Share
Do Gia Huy	Pedix Developer & Cloud Support	50%
Huynh Doan Hoang Minh	Cloud Architecture & System Support	50%

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Scope and Target Population	3
1.3	System Positioning	3
2	System Overview	3
2.1	Architectural Classification	3
2.2	High-Level Architecture	4
2.3	Design Decision: Custom Agent Loop	4
2.4	Technology Stack	5
3	AI & Agentic Reasoning Architecture	5
3.1	Agentic Reasoning Loop	5
3.1.1	Stage 0: Pediatric Safety Screen	6
3.1.2	Pre-Loop Input Routing: Content Check and Intent Classification .	7
3.1.3	Stage 1: Age Detection and Query Analysis	9
3.1.4	Stage 2: Age-Stratified Retrieval	9
3.1.5	Stage 3: Care Pathway Reasoning	10
3.1.6	Stage 4: Reflection and Completeness Check	11
3.1.7	Stage 5: Structured Output Generation	11
3.1.8	Memory and Child Profiles	12
3.2	Knowledge Base	12
3.2.1	Source Selection	12
3.2.2	Metadata and Indexing Strategy	13
3.2.3	Ingestion Pipeline	13
3.3	Safety Architecture	14
4	Cloud Infrastructure & Deployment	14
4.1	High-Level AWS Architecture	14
4.2	AWS Service Map	16
4.3	Cost Engineering Rationale	16
4.4	Cost Governance	17
5	Application Features	17
6	Project Goals and Learning Outcomes	18
7	Team Contributions	18

Abstract

PEDIX is a cloud-deployed AI health assistant that helps parents of children aged 0–5 years navigate from observed symptoms to an appropriate care pathway. Built on a *Level 4 Agentic RAG* architecture, the system features a custom five-stage reasoning loop with age-stratified clinical reasoning at its core. Unlike conventional symptom checkers that adapt adult logic for pediatric use, PEDIX is pediatric-native: its knowledge base, retrieval filters, urgency thresholds, and output tone are grounded in guidelines from the WHO, NICE, CDC, and AAP. A hybrid two-layer pre-inference safety screen detects life-threatening pediatric red flags: a deterministic rule-based layer executes in under 10 milliseconds, followed by a lightweight LLM verification pass (Claude Haiku) to eliminate context-blind false positives before any substantive inference is invoked. An input routing layer further classifies each message into triage, general knowledge, or non-clinical intent, enabling the agent to serve parents seeking general pediatric information without requiring child-specific data. The system is deployed on AWS using multiple services, with cost governance as an explicit engineering goal. PEDIX demonstrates current industry practice in applied AI—transparent, evidence-backed, and architecturally sound.

1. Introduction

1.1 Problem Statement

When a young child falls ill, parents face a high-stakes, time-sensitive decision with limited information: how serious is this, and what should they do next? This decision is complicated by two factors.

First, pediatric clinical thresholds are dramatically age-dependent. A rectal temperature of 38.0 °C in a neonate under 28 days is a medical emergency; the same temperature in a three-year-old is typically manageable at home [1]. General-purpose symptom checkers, whether rule-based or LLM-powered, routinely fail to encode this distinction, as they are built for adult populations and retrofitted with surface-level pediatric modes.

Second, parents searching online encounter information that is either too generic, too alarming, or too passive to be actionable. There is a gap between what a knowledgeable pediatric clinician would tell an anxious parent informally and what current digital tools provide.

PEDIX is designed to close that gap: not by replacing clinical judgment, but by making evidence-backed, age-stratified care navigation accessible to any parent at any hour.

1.2 Scope and Target Population

PEDIX targets children from birth through five years of age. This range corresponds to a globally recognized clinical category within which symptom severity, urgency thresholds, and appropriate care settings change substantially across five sub-groups: neonates (0–28 days), young infants (1–3 months), infants (3–12 months), toddlers (1–3 years), and preschool-aged children (3–5 years).

- **Included:** symptom-to-care-pathway navigation, urgency classification, pre-visit preparation guidance, and cited clinical evidence.
- **Excluded:** diagnosis, prescription dosing, surgical guidance, children over five, adult content, and computer vision-based assessment.

1.3 System Positioning

PEDIX occupies a deliberate position in the landscape of health AI tools:

What it is	What it is not
<i>A care pathway navigator</i>	A diagnostic tool (never outputs “your child has <i>X</i> ”)
<i>An agentic reasoning system</i>	A chatbot or fixed Q&A pipeline
<i>A dynamic knowledge base</i>	A static FAQ or hardcoded rule engine
<i>Pediatric-native & Personalized</i>	An adult tool with a child mode

The compass metaphor is intentional: a compass does not tell you what is at the destination, it points you in the right direction.

2. System Overview

2.1 Architectural Classification

PEDIX implements a **Level 4 Agentic RAG** architecture. The following table contextualizes this classification within the RAG complexity spectrum:

Level	Name	Description
1	Naive RAG	Retrieve then generate; no decision logic
2	Pipeline RAG	Fixed multi-step pipeline; code decides the steps
3	Self-RAG	Conditional retrieval; model decides <i>whether</i> to retrieve
4	Agentic RAG	Model plans steps, acts, reflects, and loops autonomously
5	Multi-Agent System	Multiple coordinating LLM agents

The critical distinction between Level 2 and Level 4 is *agency over the reasoning process*. In a pipeline, the code determines step execution regardless of necessity. In an agentic loop, the language model decides: is the child’s age established? Is the urgency assessment well-supported? Should another retrieval pass be performed? The agent plans, acts, observes, and decides what to do next.

2.2 High-Level Architecture

The system is organized into four logical tiers: presentation, API, application, and data – as illustrated in Figure 3. The agentic reasoning loop at the application tier is the core of the system; all surrounding infrastructure exists to serve it reliably and cost-effectively.

2.3 Design Decision: Custom Agent Loop

PEDIX implements its reasoning loop in pure Python without relying on frameworks such as LangChain or LlamaIndex. This is deliberate: the age-stratification logic, retrieval filter construction, and reflection step must be explicitly visible in code for portfolio demonstration, debugging, and explainability in interview contexts. Additionally, Stage 0’s deterministic layer runs in under 10ms with no LLM dependency; a secondary Haiku verification pass is invoked only when the first layer detects a potential red flag keyword, keeping the fast path latency-free for the vast majority of queries. Framework abstractions could introduce unpredictable latency or behavior in this critical safety path.

2.4 Technology Stack

Layer	Technology
Frontend	React.js, hosted via CloudFront + S3
Backend	FastAPI (Python) on EC2
Vector Database	Qdrant (Docker container on EC2)
LLM Inference	Amazon Bedrock (Claude Sonnet)
Embeddings	HuggingFace <code>all-MiniLM-L6-v2</code> (384 dims)
Reranker	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code> (HuggingFace)
Session Storage	DynamoDB
Authentication	AWS Cognito
File Storage	S3 (Frontend only)
Ingestion	Offline Python Script (Local)
Agent Framework	Custom Python loop (no LangChain / LlamaIndex)
Monitoring	Amazon CloudWatch
API Layer	AWS API Gateway
CDN	Amazon CloudFront

3. AI & Agentic Reasoning Architecture

3.1 Agentic Reasoning Loop

This section constitutes the primary technical contribution of the proposal. PEDIX processes each parent query through a structured five-stage reasoning loop (Figure 1), preceded by a two-component input routing layer that classifies intent and filters non-clinical messages before any substantive reasoning begins. Stages 1–4 may iterate up to two times before Stage 5 produces the final output.

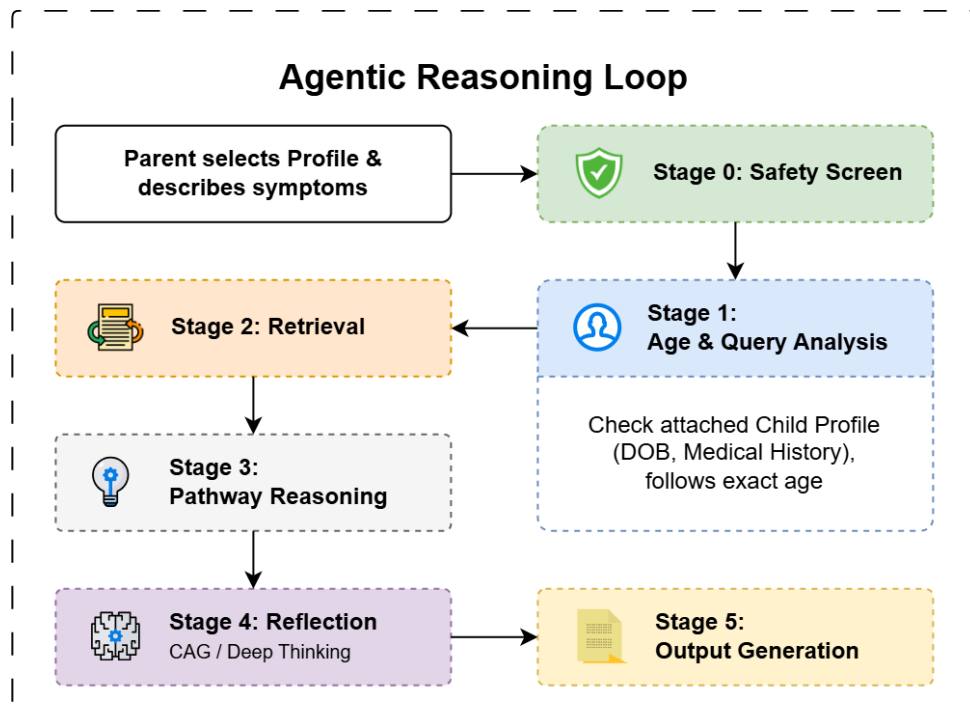


Figure 1: The full query processing pipeline. Stage 0 is a hybrid two-layer safety screen: Layer 1 applies deterministic keyword matching (no LLM); Layer 2 invokes Claude Haiku to verify emergency context only when Layer 1 triggers. Following Stage 0, an input routing layer performs a Content Check and Intent Classification before the five-stage reasoning loop begins. If a red flag is confirmed, or if the message is a greeting or general knowledge query, the system returns immediately without entering the triage loop. Stages 1–4 may loop back once if the reflection step determines the assessment is incomplete.

3.1.1 Stage 0: Pediatric Safety Screen

Stage 0 is a **hybrid two-layer screen** that executes before the reasoning loop. It is designed to catch life-threatening pediatric emergencies with both speed and contextual accuracy.

Layer 1 – Deterministic Keyword Detection (no LLM). The first layer performs rule-based pattern matching against a curated set of pediatric red flag keywords. It contains no LLM call, is immune to hallucination, and executes in under 10 ms. Red flag patterns include: any fever in a child under 90 days of age, bulging fontanelle, cyanosis, high-pitched or inconsolable cry, febrile seizure, breathing difficulty or grunting, unresponsiveness, and severe dehydration signs.

If Layer 1 finds no match, Stage 0 exits and input routing begins. If Layer 1 detects a keyword match, Layer 2 is invoked.

Layer 2 – Contextual Verification (Claude Haiku). A rule-based keyword match alone is insufficient for accurate emergency detection. A parent may mention “seizure” in a clearly non-emergency context (e.g., “my child had a seizure last month, he has been

fine since”), and a false positive would route them to an emergency response unnecessarily. Layer 2 addresses this by invoking Claude Haiku – a fast, lightweight model – to verify whether the matched keyword genuinely reflects an active or imminent emergency, given the full context of the message.

If Haiku confirms an emergency context, the system immediately returns a structured emergency response (specific trigger, recommended action, and call-to-emergency-services directive) and bypasses the reasoning loop entirely. If Haiku determines the mention is incidental, Stage 0 passes and input routing proceeds normally.

This two-layer design preserves sub-10 ms latency for the majority of queries (Layer 1 negative) while adding contextual judgment only when it is clinically necessary (Layer 1 positive).

3.1.2 Pre-Loop Input Routing: Content Check and Intent Classification

Before the five-stage reasoning loop begins, all messages that pass Stage 0 are routed through two sequential steps. Together they determine whether the query should enter the triage loop, be answered as a general knowledge request, or receive a simple greeting response. This routing layer prevents the agent from asking “how old is your child?” when the parent has simply typed “Hi”.

Step A – Content Check (rule-based, no LLM). The Content Check determines whether the message contains sufficient actionable content to warrant reasoning. It applies only to the *first message* of a session; multi-turn messages are always forwarded directly to intent classification, since prior conversation history provides sufficient context.

For first messages, the check applies the following ordered rules:

1. If the message matches a greeting pattern (e.g., “hi”, “hello”, “hey”) → return a **greeting** response.
2. If the message contains a recognized medical or pediatric keyword (e.g., “fever”, “RSV”, “seizure”, “vaccine”, “dehydration”) → pass immediately, regardless of message length.
3. If the message matches a vague help-seeking pattern (e.g., “can you help”, “what can you do”) → return a **greeting** response.
4. If the message contains fewer than three content words after removing stop words → return a **greeting** response.
5. Otherwise → pass.

The medical keyword passlist in Rule 2 is intentionally placed before the vague pattern check (Rule 3), ensuring that short but medically meaningful inputs such as “RSV?” or “paracetamol safe?” pass through correctly rather than being caught by vague pattern

matching.

No LLM is used in this step. Stakes are low (a false positive produces a friendly greeting rather than a clinical error), so deterministic rules are sufficient and preferable for speed.

Step B – Intent Classification (Claude Sonnet, tool_use). Messages that pass the Content Check are classified into one of three intent types using a structured Sonnet tool call (`max_tokens = 150`):

Intent	Description and Routing
TRIAGE	User is asking about a specific child’s current or recent symptoms. Routes to the full five-stage reasoning loop (Stage 1 onward).
GENERAL	User is seeking general pediatric health knowledge (“what is RSV?”, “how long does a cold last?”). Routes to a dedicated General RAG handler: retrieval without age_group filter, prose response, cited sources, and a soft nudge inviting the parent to share their child’s specific situation if applicable.
HIGH_STAKES_GENERAL	A GENERAL query about a potentially life-threatening topic (seizure, breathing difficulty, cyanosis, meningitis). Before answering, the agent returns a confirmation response with two quick-reply buttons: “ <i>My child is experiencing this right now</i> ” (routes to TRIAGE) or “ <i>I’m asking to learn in general</i> ” (routes to GENERAL).

When intent is ambiguous and stakes are low, the classifier defaults to **GENERAL** to minimize friction. When ambiguous and stakes are high, it defaults to **HIGH_STAKES_GENERAL**. A message referencing a specific child (“my baby”, “my son”) combined with a current symptom is always classified as **TRIAGE**, even if phrased as a question.

General RAG response design. For **GENERAL** intent, retrieval queries the full knowledge base without an **age_group** pre-filter, since the question is not specific to a particular child. The entire knowledge base is exclusively pediatric under-5 content, so an unfiltered search does not risk surfacing adult material. Where retrieved chunks reflect age-dependent variation, the response naturally incorporates this (e.g., “RSV is particularly dangerous in children under three months”). Each response closes with a fixed soft nudge encouraging the parent to share their child’s specific situation for more personalized guidance.

Analytics. All intent classifications – including **greeting**, **general**, **confirmation**, **triage**, and **emergency** – are logged to the analytics store. This enables the Analytics Dashboard (Module 3) to display an **Intent Distribution** chart, providing operational

insight into how parents are actually using the system.

3.1.3 Stage 1: Age Detection and Query Analysis

Age is the single most clinically consequential variable in pediatric triage. Stage 1 executes only for **TRIAGE**-classified queries. The agent’s first priority on reading any parent message is to establish the child’s age and clinical context.

The agent first checks if the user has attached a specific **Child Profile** to the chat session. If a profile is attached, the system dynamically calculates the exact age in days from the Date of Birth (DOB) and injects the child’s underlying medical history and weight directly into the LLM context. This context-first approach dramatically increases the clinical safety and accuracy of the urgency assessment. If no profile is attached, the agent immediately asks for the child’s age before any other follow-up question. Once established, the age is mapped to one of five clinical groups:

Age Group	Range	Clinical Significance
Newborn	0–28 days	Highest risk; distinct immune and thermal physiology
Young Infant	1–3 months	High-risk; immature immune response
Infant	3–12 months	Transitional; threshold-dependent urgency
Toddler	1–3 years	More robust; standard pediatric thresholds
Preschool	3–5 years	Closer to standard child thresholds

The practical consequence of this classification is significant. As shown in Table 1, the same symptom at different ages produces categorically different urgency assessments.

Table 1: Age-stratified urgency classification for selected symptoms

Symptom	Newborn	Infant	Toddler	Preschool
Fever 38.0°C	Emergency	Urgent	Monitor	Monitor
Not feeding well	Emergency	Urgent	Soon	Routine
Vomiting (once)	Urgent	Monitor	Routine	Routine
Mild cough	Urgent	Soon	Routine	Routine

After resolving age, the agent determines what additional information is needed for a well-grounded recommendation, such as exact temperature, symptom duration, feeding status, vaccination history, or prematurity status, then generates 1–3 targeted follow-up questions. If all critical information is already present, the agent proceeds directly to Stage 2.

3.1.4 Stage 2: Age-Stratified Retrieval

PEDIX uses Qdrant as its vector store. Every document chunk is indexed with structured metadata including `age_group`, `urgency_relevance`, `source_authority`, `content_type`, and `publication_date`. The retrieval pipeline operates in two passes, as illustrated in

Figure 2.

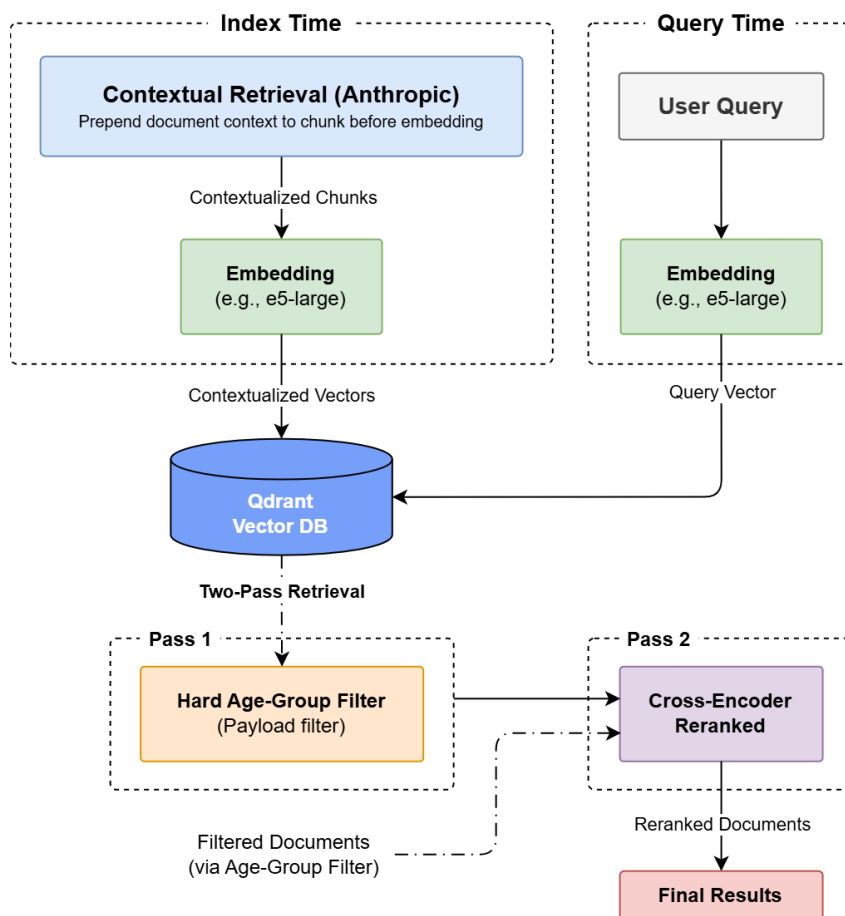


Figure 2: Two-pass age-stratified retrieval pipeline. Pass 1 applies a hard `age_group` filter in Qdrant to ensure only age-appropriate chunks are considered. Pass 2 applies a cross-encoder reranker scoring by symptom overlap, age specificity, and source authority. The top-3 reranked chunks are passed to LLM reasoning stages.

Pass 1 (Hard Filtering): The `age_group` field functions as a mandatory pre-filter applied before vector similarity search. Only chunks tagged for the child’s specific age group (or tagged “all”) are included in the candidate set. Without this filter, a chunk describing a four-year-old fever protocol could be retrieved when reasoning about a six-day-old neonate – a clinically dangerous retrieval error.

Pass 2 (Reranking): A cross-encoder reranker re-scores the age-filtered candidates by the combination of symptom overlap, age specificity, and source authority weight. The final top-3 chunks from the reranked set are passed to the LLM stages.

3.1.5 Stage 3: Care Pathway Reasoning

The agent reasons over retrieved chunks to produce a structured pathway assessment. For each query, it determines: (1) an **urgency level** using the Emergency Severity Index (ESI v4) framework: Emergency, Urgent, Soon, Routine, or Self-care; (2) the **appropriate care setting**: Pediatric ED, Urgent Care, Pediatrician appointment, or Home monitoring;

and (3) **supporting clinical evidence** from the retrieved guideline passages.

The ESI v4 urgency classification algorithm is encoded directly into the Stage 3 system prompt rather than retrieved from the vector store, preventing retrieval misses on the urgency framework itself while keeping pediatric-specific clinical thresholds in the retrieval-augmented layer.

Amazon Bedrock (Claude Haiku) serves as the LLM provider for all reasoning stages. Haiku was ultimately selected over Sonnet due to rate limit constraints and model validation issues with Sonnet during live deployment. Per-stage token budgets enforce cost predictability: 300 tokens for analysis, 500 for pathway reasoning, 300 for reflection, and 800 for output generation.

3.1.6 Stage 4: Reflection and Completeness Check

Before generating the parent-facing response, the agent performs a structured self-evaluation of the care pathway draft. Applying **Cognitive Agentic Generation (CAG)** deep thinking principles [6], it assesses whether the urgency level is consistent with the child’s age-group thresholds, whether the correct clinical norms were applied, whether there are age-specific red flags that have not been explicitly addressed, and whether a more specific guideline exists for the given age-symptom combination.

If the draft is incomplete or internally inconsistent, the agent loops back to Stage 1 with a more targeted internal query. This loop is bounded at two iterations maximum to prevent runaway inference costs.

3.1.7 Stage 5: Structured Output Generation

The final stage produces a parent-facing response with five components:

1. **Urgency badge:** color-coded level (Emergency / Urgent / Soon / Routine) displayed prominently.
2. **Care pathway:** recommended care setting, timing, and immediate actions.
3. **Pre-visit checklist:** vaccination record, symptom timeline, temperature log, and suggested questions for the pediatrician.
4. **Warning signs to monitor:** age-appropriate home monitoring guidance.
5. **Cited sources:** every recommendation linked to the specific clinical guideline chunk from which it was derived.

A collapsible reasoning trace panel is also surfaced in the UI, exposing every decision the agent made throughout Stages 0–4. The output tone is calibrated for anxious parents: clear, calm, specific, and actionable.

3.1.8 Memory and Child Profiles

PEDIX maintains conversational memory and persistent user data to improve safety and reduce friction for parents.

Child Profile Management: Parents can proactively save child profiles containing a Nickname (to protect personally identifiable information), Date of Birth, Gender, Weight, and Underlying Medical Conditions. Storing DOB rather than age ensures the system calculates the exact age dynamically at runtime, preventing dangerous “stale age” errors. To address data staleness for transient metrics like weight, the system tracks a `last_updated` timestamp. If a profile is older than 30 days upon a return visit, an optional, non-blocking UI pop-up reminds the parent to update the profile.

Session Memory: The system exclusively uses ephemeral **Within-session memory** for conversation history, which is cleared upon session termination to simplify state management and protect privacy. Cross-session chat history is explicitly excluded from the project scope.

3.2 Knowledge Base

3.2.1 Source Selection

The knowledge base is curated exclusively from established, publicly accessible pediatric clinical sources. Adult content is explicitly excluded.

Table 2: Knowledge base sources and estimated scale

Source	Org.	Age	Role	Est. Chunks
WHO Pocket Book	WHO	0–5y	Triage, emergency assessment, illness protocols	~800
Fever in Under 5s (CG160)	NICE	0–5y	Evidence-based fever triage; traffic light system	~150
Pediatric Protocols	CDC	0–5y	Vaccine schedules, milestones, illness guidance	~250
HealthyChildren.org	AAP	0–5y	Parent-facing symptom guidance	~300
Bright Futures	AAP/HRSA	0–5y	Health supervision guidelines	~200
ESI v4 (algorithm only)	AHRQ	All	Urgency classification framework (in Stage 3 prompt)	~80
Total estimated				~1,780

Approximately 1,800 chunks represents the target sweet spot: sufficient coverage for broad symptom queries while compact enough to maintain retrieval precision. At 384 dimensions, this fits comfortably in Qdrant on a t3.micro EC2 instance.

3.2.2 Metadata and Indexing Strategy

Every chunk in Qdrant is tagged with five metadata fields: **age_group** (newborn, young infant, infant, toddler, preschool, or all), **urgency_relevance** (emergency, urgent, or routine), **source_authority** (WHO, NICE, CDC, or AAP), **content_type** (triage_protocol, symptom_cluster, care_pathway, or parent_education), and **publication_date**. The **age_group** tag is the most critical, it is applied as a pre-filter before vector similarity search, ensuring the agent never retrieves content intended for the wrong age population.

3.2.3 Ingestion Pipeline

New clinical guidelines are processed through an offline local pipeline to simplify the AWS architecture. An admin runs a Python script locally that performs five sequential operations: (1) document parsing, (2) semantic chunking, (3) **Contextual Retrieval pre-processing** [7] (where a local LLM prepends a document-level summary to each chunk to prevent context loss), (4) metadata auto-tagging, and (5) embedding via HuggingFace **all-MiniLM-L6-v2**. The script pushes the resulting vectors directly to the production Qdrant EC2 instance via its REST API.

An admin metadata editor allows manual override of any auto-assigned tag before or after indexing.

3.3 Safety Architecture

PEDIX implements three independent, layered safety mechanisms. The layered design ensures that a failure in any single layer does not compromise patient safety.

Layer 1 – Pre-Loop Pediatric Emergency Screen (Hybrid). A two-step hybrid screen described in full in Section 3.1. Step 1 performs hard-coded deterministic keyword matching with no LLM dependency, executing in under 10 ms and immune to hallucination. Step 2 invokes Claude Haiku for contextual verification only when Step 1 detects a keyword match, preventing false positives caused by incidental mention of emergency-related terms. If an emergency is confirmed, the system returns an immediate emergency response and bypasses the reasoning loop entirely.

Layer 2 – LLM System Prompt Constraints. Every Bedrock call includes a system prompt that prohibits diagnosis language (*“your child has...”*, *“this is...”*), requires age-appropriate clinical thresholds to be explicitly referenced, mandates a disclaimer to consult a pediatric healthcare professional, and prohibits minimizing language for children under three months.

Layer 3 – Post-Generation Output Validator. A pattern scan on the final generated output before it is returned to the parent. If diagnosis-pattern language is detected, the output is replaced with a pre-authored safe fallback response and the event is logged to CloudWatch for review.

4. Cloud Infrastructure & Deployment

4.1 High-Level AWS Architecture

The system’s cloud architecture is designed for zero-trust security, cost-efficiency, and highly decoupled networking. The core application runs within a private subnet on an EC2 instance (`t3.micro`) inside the Default VPC.

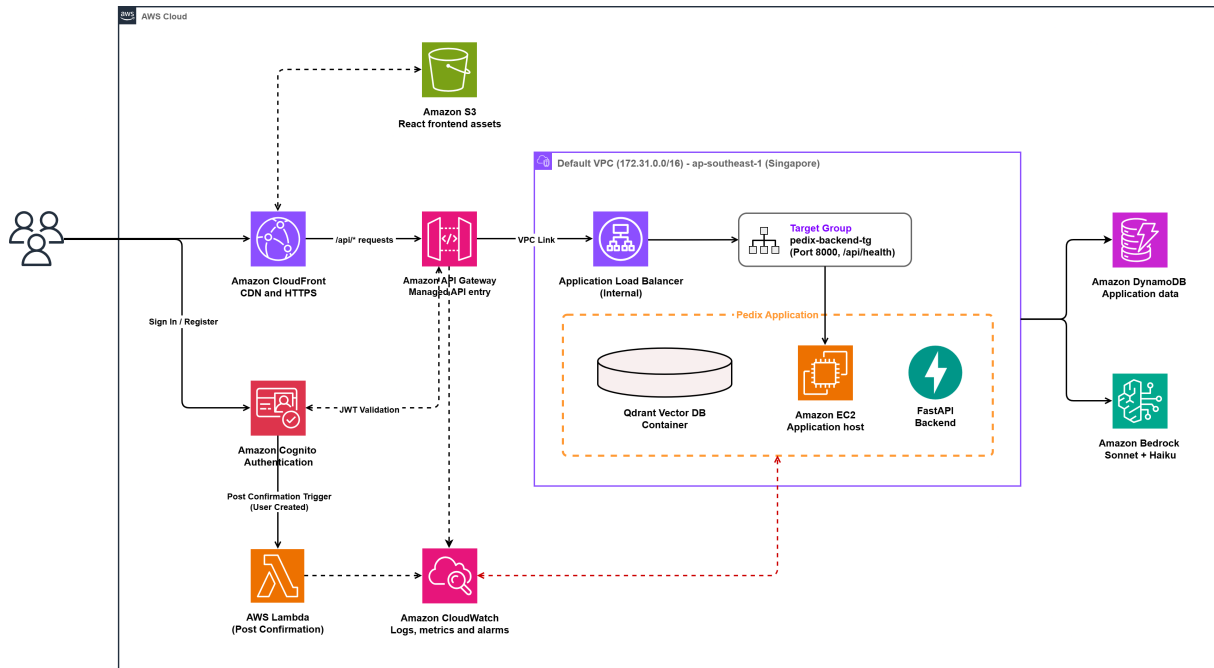


Figure 3: Pedix system architecture. The four-tier design separates concerns between client delivery (CloudFront/S3), API management (API Gateway/Cognito), application logic (FastAPI/Agent Loop/Bedrock/Qdrant), and data persistence (DynamoDB/Offline local ingestion).

Inbound Traffic & Security: All client requests are served through Amazon CloudFront, which caches static frontend assets from a private S3 bucket. Dynamic API requests are routed to a Regional API Gateway. The API Gateway authenticates requests via a Cognito JWT Authorizer. Validated requests are securely tunneled into the VPC via a VPC Link V2, hitting an Internal Application Load Balancer (ALB). The EC2 instance’s Security Group is strictly configured to only accept port 8000 inbound traffic originating from the ALB’s Security Group, completely shielding it from direct internet access.

IAM Roles & Secure Communication: Although the EC2 instance resides in a private subnet, it requires access to managed AWS services like DynamoDB (for session and profile storage) and Amazon Bedrock (for Claude Haiku inference). Instead of hardcoding access keys or storing credentials in environment variables, the EC2 instance is attached to an **IAM Instance Profile** (IAM Role). This role grants the instance explicit, least-privilege permissions (`dynamodb:PutItem`, `bedrock:InvokeModel`, etc.). When the FastAPI backend (using `boto3`) attempts to communicate with DynamoDB or Bedrock, it automatically and seamlessly acquires temporary credentials from the instance metadata service. This architecture ensures highly secure, keyless authentication between internal AWS services and prevents credential leakage.

Containerization & Statefulness: The Qdrant vector database runs locally as a Docker container on the same EC2 instance, bound exclusively to `localhost:6333`. This eliminates the need for expensive managed vector database clusters like OpenSearch

Serverless, while ensuring that vector data cannot be accessed from outside the host machine.

Real-Time Streaming Optimization: To support Agentic RAG latency without dropping connections, the FastAPI server utilizes Uvicorn with a single worker process (`--workers 1`) to maintain synchronized state for Server-Sent Events (SSE) connections in the `PendingRequestStore`. To prevent the Internal ALB and API Gateway from dropping idle connections during deep LLM reasoning cycles, the orchestrator emits a heartbeat event every 5.0 seconds.

4.2 AWS Service Map

PEDIX is deployed on Amazon Web Services using twelve services, chosen to balance functionality against cost constraints within the program budget.

Table 3: AWS services used in Pedix

#	Service	Role	Est. Cost/mo
1	EC2 (t3.micro)	FastAPI backend + Qdrant (Docker)	~\$16.15
2	S3	React frontend assets	Free tier
3	DynamoDB	Child profiles, document registry	Free tier
4	API Gateway	HTTPS REST layer (VPC Link V2)	~\$18.25
5	Amazon Bedrock	LLM inference (Claude Haiku) for all stages	~\$14.00
6	Cognito	User authentication and role authorization	Free tier
7	CloudWatch	Logs, metrics, billing alarms	Free tier
8	CloudFront	CDN delivery of React frontend	Free tier
9	Application Load Balancer	HTTPS termination + EC2 traffic routing	~\$24.24
Estimated total monthly cost			~\$72.70

4.3 Cost Engineering Rationale

Several infrastructure choices were made explicitly to control cost:

- **DynamoDB over RDS:** RDS requires a minimum provisioned instance at \$15–30/month. DynamoDB on pay-per-request is effectively free at student project volumes, while natively supporting the relational `ChildProfiles` table mapped to `Cognito UserID`.
- **Qdrant on EC2 over OpenSearch Serverless:** OpenSearch Serverless requires two always-on OCUs at \$50–70/month—enough to consume a large portion of the budget. Qdrant runs as a Docker container on the same EC2 instance at no marginal cost, with

full metadata filtering support.

- **Claude Haiku over Sonnet/Opus:** Haiku provides sufficient reasoning depth for age-stratified inference while maintaining strict rate limits and predictable costs.
- **Per-stage token budgets:** Maximum token counts enforced per inference stage prevent cost overruns while preserving output quality.

4.4 Cost Governance

Cost monitoring is configured from deployment Day 1:

- AWS Budgets with SNS email alerts at \$30, \$60, and \$100 thresholds.
- CloudWatch billing alarm at \$50 monthly spend.
- Every resource tagged **Project: Pedix** for per-service cost breakdown.
- EC2 scheduling: automatic stop outside development hours.
- Qdrant collection size capped at 50 MB during development.

5. Application Features

The final deliverable is a full-stack web application consisting of four modules.

Module 1 – Chat Interface (Core). The primary interface where parents describe symptoms in natural language. Key design elements include: a **Profile Selector** allowing parents to attach a specific child’s profile to the session (triggering an optional 1-month update reminder if data is stale), a follow-up question flow with quick-tap response options, a color-coded urgency badge, a collapsible reasoning trace panel, cited source links, and a downloadable pre-visit checklist. The interface is **mobile-first**, parents search symptoms on phones, often under stress, at night.

Module 2 – Knowledge Base Manager (Admin). A collection of local Python scripts used by the administrative team to process and upload new clinical guidelines directly to the vector database. This offline approach eliminates the need for complex serverless ingestion pipelines on AWS.

Module 3 – Analytics Dashboard. Operational metrics powered by CloudWatch and frontend charts: total queries by period, **intent distribution** (triage / general / greeting / emergency), urgency level distribution, most-queried symptom categories, age group distribution, average reasoning loop iterations, and retrieval confidence scores.

Module 4 – User Authentication. AWS Cognito-powered login and registration. Authenticated users can save Child Profiles. Anonymous mode available for single-session use.

6. Project Goals and Learning Outcomes

PEDIX serves four concurrent goals:

1. **Portfolio demonstration.** Age-stratified Level 4 Agentic RAG with visible reasoning trace, multi-layer safety architecture, and cited pediatric sources represents current industry practice in applied AI (2025–2026).
2. **AWS cloud proficiency.** Twelve services, cost governance, resource tagging, billing alarms, and documented trade-offs demonstrate practical cloud engineering.
3. **Responsible AI design.** Pediatric-native guardrails, mandatory citation, diagnosis prohibition, and a deterministic safety screen demonstrate awareness of ethical obligations in health-adjacent AI.
4. **Engineering discipline.** Custom agent loop, semantic chunking with metadata filtering, cross-encoder reranking, and per-stage token budgets demonstrate considered architectural decision-making.

7. Team Contributions

Table 4: Team member contributions

Member	Share	Contributions
Do Gia Huy	50%	Pedix Developer & Cloud Support - Responsible for the Main Agentic System
Huynh Doan Hoang Minh	50%	Cloud Architecture & System Support - Design the Cloud Architecture System

References

- [1] National Institute for Health and Care Excellence (NICE). *Fever in under 5s: assessment and initial management. Clinical Guideline CG160*. NICE, London, 2013 (updated 2019). Available at <https://www.nice.org.uk/guidance/ng143>
- [2] World Health Organization. *Pocket Book of Hospital Care for Children: Guidelines for the Management of Common Childhood Illnesses*, 2nd ed. WHO Press, Geneva, 2013. Available at <https://www.who.int/publications/i/item/9789241548373>

-
- [3] American Academy of Pediatrics. *HealthyChildren.org: Pediatric Symptom Guidance*. AAP, 2024. Available at <https://www.healthychildren.org>
 - [4] Centers for Disease Control and Prevention. *Child and Adolescent Immunization Schedule by Age*. CDC, Addendum, 2025. Available at <https://www.cdc.gov/vaccines/hcp/imz-schedules/child-adolescent-age.html>
 - [5] Gilboy N., Tanabe P., Travers D., Rosenau A.M. *Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4*. AHRQ Publication No. 12-0014. Agency for Healthcare Research and Quality, Rockville, MD, 2011.
 - [6] Li, J., et al. *CAG: Cognitive Agentic Generation for Deep Thinking Systems*. ACM Transactions on AI, 2025.
 - [7] Anthropic. *Contextual Retrieval: Improving RAG with document context*. Anthropic Engineering Blog, 2024. Available at <https://www.anthropic.com/engineering/contextual-retrieval>
 - [8] Hagan J.F., Shaw J.S., Duncan P.M. (Eds.). *Bright Futures: Guidelines for Health Supervision of Infants, Children, and Adolescents*, 4th ed. American Academy of Pediatrics, Elk Grove Village, IL, 2017.
 - [9] Lewis P., Perez E., Piktus A., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 2020.
 - [10] Shinn N., Cassano F., Labash B., Gopinath G., Narasimhan K., Yao S. *Reflexion: Language Agents with Verbal Reinforcement Learning*. Advances in Neural Information Processing Systems (NeurIPS), 2023.